3. Processos e Threads

"Nunca estou realmente satisfeita por entender algo; porque, por mais que eu entenda bem, minha compreensão só pode ser uma fração infinitesimal de tudo o que quero entender."

– Ada Lovelace, *1815 — 1852*

As unidades fundamentais de execução em um sistema operacional são os processos e as *threads*. Ambos podem ser iniciados, suspensos, ter sua prioridade alterada ou terminados pelo sistema operacional, competindo pelo uso do processador conforme algoritmos de escalonamento pré-definidos. Neste capítulo, exploraremos em detalhes os processos e *threads*, seus mecanismos de escalonamento e as formas de comunicação entre eles.

3.1 TRADUZINDO OS PROGRAMAS

Antes de iniciarmos o estudo dos processos e *threads*, vamos entender como um programa escrito em **linguagem de alto nível** (que é de fácil compreensão para os programadores) é transformado em um programa executável em **linguagem de máquina** (conjunto de instruções que é compreensível pelo processador). Cada tipo de processador possui uma linguagem de máquina específica, um dialeto próprio, para o qual os programas em alto nível precisam ser traduzidos antes de serem executados.

É importante notar que os procedimentos aqui descritos não se aplicam às linguagens interpretadas, como Java e Python, que se utilizam de máquinas virtuais, conforme veremos na Seção 9.2.

O arquivo escrito em **linguagem de alto nível** é convertido em várias etapas pelo compilador, resultando em um arquivo em **linguagem de máquina**. Essas diversas etapas permitem que a tradução seja feita de modo mais eficiente e garantem que o próprio compilador e o conjunto de ferramentas associadas possam ser adaptados mais facilmente ao trabalhar com diferentes arquiteturas de processadores.

Exemplo 3.1 – Exemplo de programa em alto nível

```
#include <stdio.h>
int main () {
        printf ("Alô mundo! \n");
}
```

DRAFT

No Exemplo 3.1 apresentamos um programa em **linguagem de alto nível** ("C") que imprime uma mensagem na saída padrão, que pode ser um terminal em uma janela na sua interface de usuário.

Primeiramente, o **compilador**, que é específico para cada linguagem de alto nível, faz a tradução para a linguagem de montagem do processador utilizado no computador. A **linguagem de montagem** (*assembly language*) é uma forma intermediária de representação em baixo nível do programa, mas ainda legível, compreensível e modificável pelo programador.

Apresentamos no Exemplo 3.2 um trecho da tradução do Exemplo 3.1 para a linguagem de montagem dos processadores da família x86, onde também encontramos várias diretivas para o programa montador.

Exemplo 3.2 – Exemplo de programa em linguagem de montagem

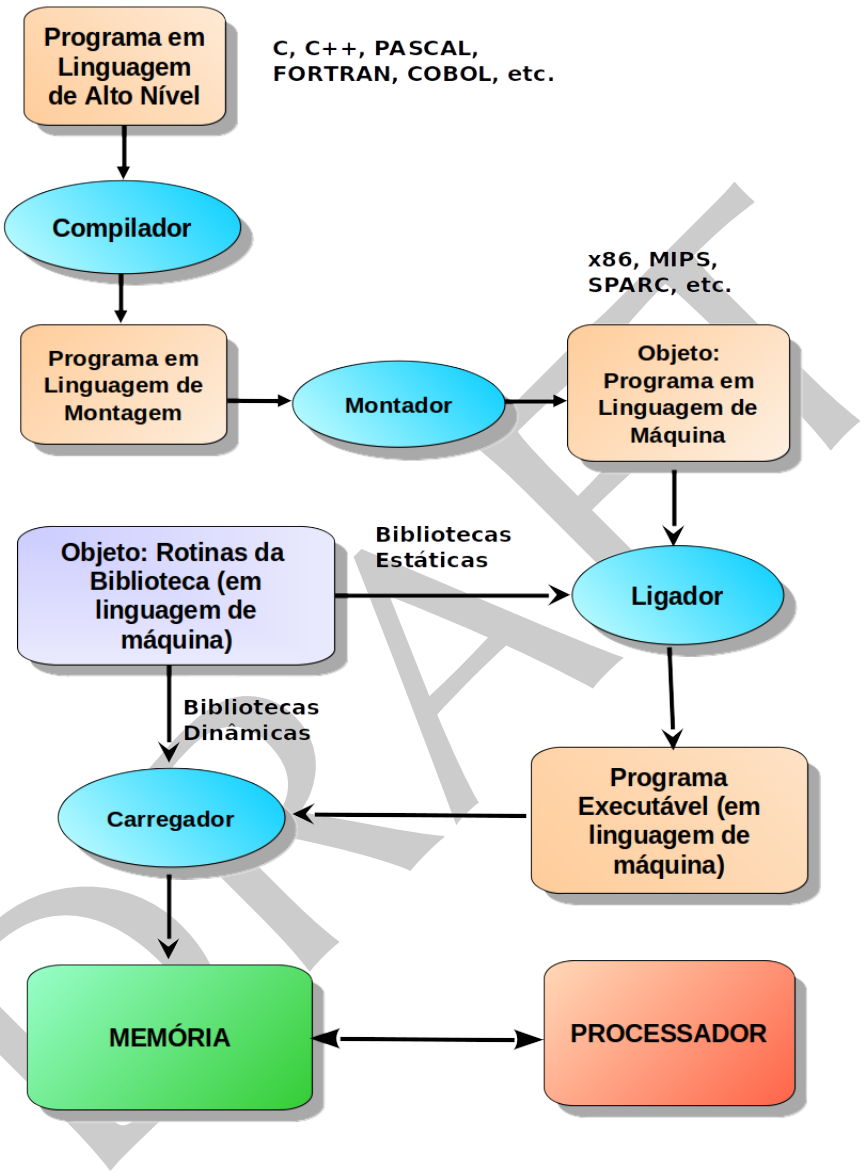
```
        .file   "teste4.c"
        .text
        .section        .rodata
.LC0:
        .string "Al\303\264 mundo! "
        .text
        .globl  main
        .type   main, @function
main:
.LFB0:
        .cfi_startproc
        endbr64
        pushq   %rbp
        .cfi_def_cfa_offset 16
        .cfi_offset 6, -16
        movq    %rsp, %rbp
        .cfi_def_cfa_register 6
        leaq    .LC0(%rip), %rax
        movq    %rax, %rdi
        call    puts@PLT
        movl    $0, %eax
        popq    %rbp
        .cfi_def_cfa 7, 8
        ret
        .cfi_endproc
```

A linguagem de montagem utiliza mnemônicos para representar as instruções da máquina, tornando o código mais legível para os programadores. Após a geração do arquivo com o código em linguagem de montagem, o **montador** (*assembler*) entra em ação, traduzindo-o para a **linguagem de máquina**. Cada instrução em linguagem de montagem geralmente corresponde a uma única instrução em máquina, mas otimizações podem gerar mais de uma instrução de máquina para uma única instrução de montagem.

DRAFT

É importante ressaltar que o conjunto de instruções e o formato da linguagem de máquina são específicos para cada tipo de processador, resultando que, para diferentes processadores, existem diferentes instruções em linguagem de montagem e de máquina.

Figura 3.1 – Fluxo de Compilação



Esse arquivo em linguagem de máquina gerado pelo montador é denominado **programa objeto**. Contudo, o programa objeto ainda não está pronto para ser executado, pois necessita da adição de um conjunto de rotinas pré-definidas, presentes nas bibliotecas da linguagem de programação. Um exemplo clássico são as funções como '*printf()*' e '*scanf()*' na linguagem 'C'. As rotinas que são explicitamente referenciadas pelo programador no código fonte são incorporadas ao programa executável pelo **ligador** (*linker*) durante esta etapa da compilação.

Em realidade, a incorporação dessas rotinas pode ocorrer de duas formas: por meio de bibliotecas estáticas ou dinâmicas. No caso das **bibliotecas estáticas**, as rotinas são integradas permanentemente ao **programa executável** pelo **ligador** (*linker*), resultando em um arquivo

executável maior do que o programa objeto original. Entretanto, essa abordagem pode levar a redundâncias, já que diferentes executáveis que utilizem as mesmas rotinas podem carregá-las várias vezes na memória, causando um desperdício desnecessário de espaço em memória.

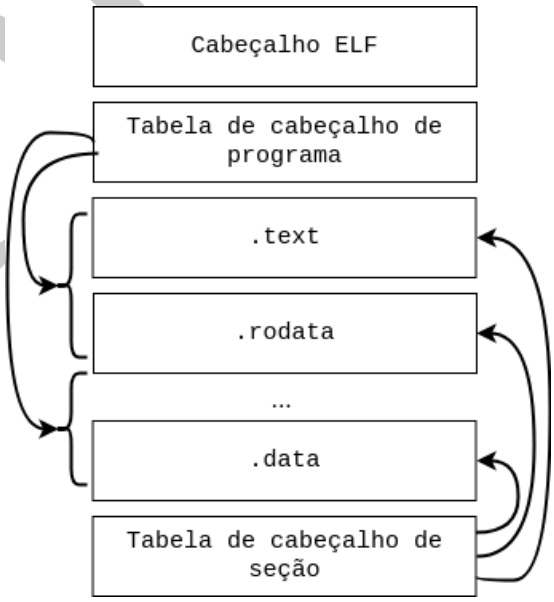
Para amenizar esse problema, utilizam-se as **bibliotecas dinâmicas**, nas quais o código das rotinas é carregado apenas uma vez na memória e compartilhado entre todos os programas que delas necessitem. Esse processo é gerenciado pelo **carregador** (*loader*) no momento em que o programa executável é carregado na memória para execução pelo processador. Ou seja, o carregador verifica se existem referências ainda indefinidas no programa a ser executado, faz a ligação com as bibliotecas dinâmicas residentes na memória ou faz a carga na memória, caso ainda não estejam presentes.

Se ao final deste processo houver ainda alguma referência externa que não possa ser resolvida, o carregador emite uma mensagem de erro e cancela a execução do programa.

É importante destacar que o processador só pode executar programas em linguagem de máquina que estejam carregados na memória e que já tenham incorporado todas as rotinas de biblioteca, seja de maneira estática ou dinâmica, ao código executável. O fluxo de tradução de um programa em linguagem de alto nível até um programa executável é ilustrado na Figura 3.1.

O **programa executável** resultante pode ter vários formatos possíveis, dependendo do sistema operacional em uso. Um dos formatos padronizados atualmente é o ELF (*Executable and Linking Format*), em versões de 32 e 64 bits, utilizado em programas executáveis, programas objeto, bibliotecas compartilhadas e despejos de memória (*core dumps*) [1]. Um diagrama esquemático deste formato pode ser visto na Figura 3.2.

Figura 3.2 – Formato ELF



Cada arquivo ELF é composto de um cabeçalho ELF, que contém informações gerais sobre o arquivo, como tipo (executável, objeto etc.), arquitetura, endereço de entrada e deslocamen-

[1] https://refspecs.linuxfoundation.org/elf/elf.pdf

DRAFT

tos para outras seções. Este cabeçalho é seguido pelos dados do arquivo. Os dados podem incluir:

1. Uma tabela de cabeçalho de programa, que descreve zero ou mais segmentos de memória, que informa ao sistema operacional como criar um processo para execução do programa;
2. Uma tabela de cabeçalhos de seção, que descreve zero ou mais seções;
3. Os dados referenciados pelas entradas na tabela de cabeçalhos de programa ou na tabela de cabeçalhos de seção.

Tanto as seções quanto os segmentos contêm a maior parte dos dados de um arquivo objeto, incluindo: instruções, dados, tabela de símbolos, informações de relocação e informações de ligação dinâmica. Qualquer byte, no arquivo inteiro, pode ser propriedade no máximo de uma seção e pode haver bytes órfãos, que não são propriedade de nenhuma seção.

O que acontece quando o sistema operacional carrega e executa um programa executável neste formato? Vamos procurar descrever a seguir, de uma maneira resumida, como é esse passo a passo.

Primeiro, procura-se o cabeçalho do programa para verificar como carregar a imagem do programa na memória. Em seguida, localiza-se a seção **.text**, que contém as instruções do programa, e a seção **.rodata**, que contém dados somente de leitura, carregando-as em páginas apenas de leitura na memória. Depois, a seção **.data** é analisada, e os dados estáticos, com valores iniciais, são carregados em páginas de leitura e escrita no espaço do usuário. O próximo passo é buscar a seção **.bss**, que contém dados estáticos sem valor inicial definido, e adicionam-se as páginas apropriadas na memória, normalmente zerando seus valores. Adicionalmente, pode haver uma seção **.symtab**, que contém a tabela de símbolos para auxiliar na depuração do programa.

Posteriormente, durante a execução do programa, as variáveis alocadas dinamicamente, por exemplo com a função *malloc()*, têm seu espaço de endereçamento reservado em uma área de memória específica para isso, de tamanho variável, chamada de ***heap***. Finalmente, outra área que cresce dinamicamente durante a execução do programa, a **pilha** (*stack*), é utilizada para armazenar informações como parâmetros de entrada e saída para as rotinas, endereços de retorno e variáveis locais às rotinas e funções. Por razões históricas, a pilha e a *heap* crescem em sentidos opostos da memória, isto é, uma do começo para o fim da memória (*heap*) e a outra do final para o início da memória (pilha).

Após ser carregado na memória e ter sua execução iniciada, o programa passa a ser conhecido como um **processo**, cujos detalhes iremos descrever a seguir.

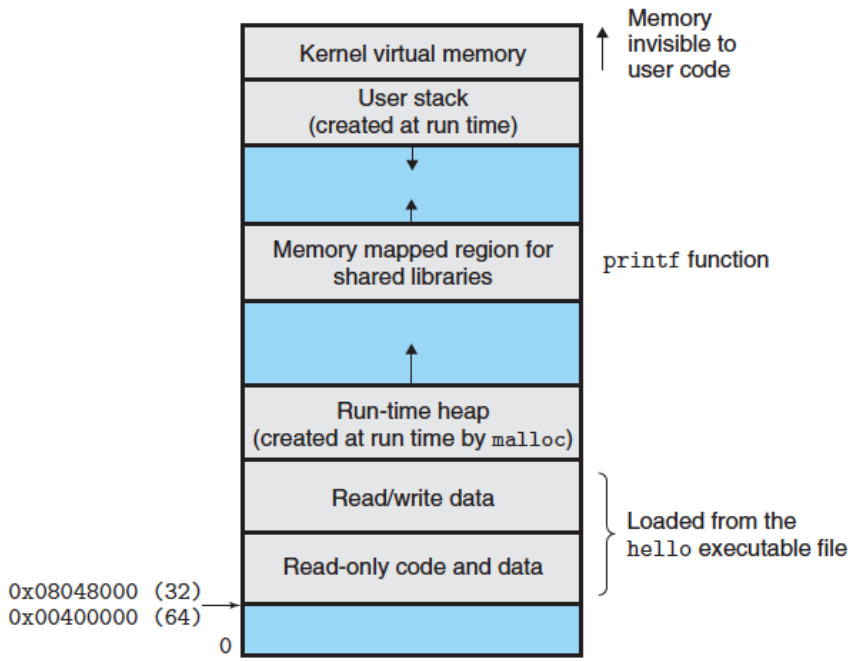
DRAFT

3.2 PROCESSOS

3.2.1 Definição

Os processos têm como objetivo realizar um conjunto de tarefas especificadas nas instruções do seu código. Enquanto os programas são entidades estáticas, os processos são dinâmicos. No entanto, um processo em um sistema operacional é mais do que apenas um programa em execução. Ele representa uma entidade que pode ser controlada e gerenciada pelo sistema operacional. Os processos podem ser iniciados, ter sua prioridade alterada, ser suspensos ou terminados arbitrariamente pelo usuário e disputam o uso do processador entre si, de acordo com os algoritmos de escalonamento do sistema operacional (veja a Seção 3.3).

Figura 3.3 – Espaço de endereçamento do processo

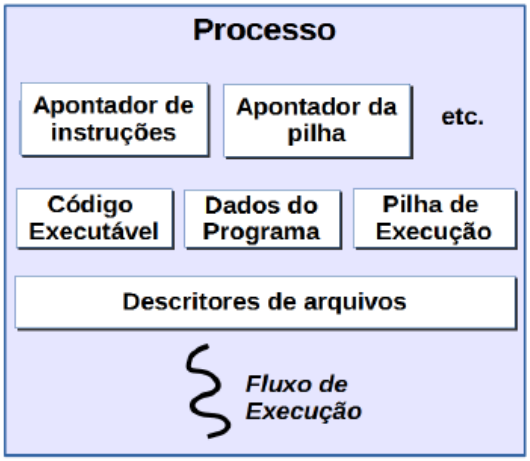


Cada processo possui seu próprio espaço de endereçamento, uma região de memória virtual dedicada que contém o código em linguagem de máquina e os dados necessários para a execução do programa, como variáveis globais, constantes e estruturas de dados estáticas e dinâmicas. Além do espaço de endereçamento, o processo está associado a um conjunto de recursos do sistema operacional, incluindo os registradores do processador, como o apontador de instruções e o ponteiro de pilha, que são usados pelo processador para controlar a execução do programa. O processo também mantém uma lista de arquivos abertos e outros recursos de entrada e saída (E/S), utilizados pelo programa para acessar dispositivos de armazenamento ou comunicação.

Além disso, os processos podem se comunicar com outros processos e com o sistema operacional através de diversos mecanismos. Uma forma de comunicação é o uso de sinais, indicando eventos como interrupções de *hardware*, solicitações de término ou outros eventos importantes. Maiores detalhes podem ser vistos na Seção 4.1.

É comum também que os processos estejam organizados em estruturas de dados que representam suas relações uns com os outros. Isso pode incluir informações sobre processos pais e filhos, processos irmãos ou processos em execução em paralelo. Um processo também inclui informações sobre seu estado: se está em execução, pronto, bloqueado, suspenso ou terminado. Além disso, pode haver informações sobre a prioridade de execução, política de escalonamento e outras características de gerenciamento de processo. Um processo é na essência um contêiner que armazena todas as informações necessárias para a execução de um programa pelo sistema operacional.

Figura 3.4 – Processo



3.2.2 Bloco de controle de processo - PCB

Cada processo possui uma estrutura de dados associada chamada de **bloco de controle de processo** (PCB), que é utilizada pelo sistema operacional para armazenar todas as informações sobre um processo, sendo também é conhecido pelo nome **descritor de processo**.

Quando um processo é criado, o sistema operacional cria um bloco de controle de processo correspondente, que contém informações de identificação, de estado e de controle do processo. Os dados de **identificação do processo** incluem um identificador exclusivo para o processo (PID – um número inteiro usualmente) e, dados como o identificador do processo pai, identificador do usuário, grupo de usuários, entre outros.

Os dados de **estado do processo** definem o *status* de um processo quando ele é suspenso, permitindo que o sistema operacional o reinicie posteriormente. O PCB especifica o estado propriamente dito do processo, ou seja, novo, pronto, em execução, bloqueado, suspenso ou encerrado. Ainda, no caso de processo suspenso, os dados de identificação do evento que o processo está aguardando devem ser armazenados. Além disso, estado do processo inclui o conteúdo dos registradores de uso geral do processador, a palavra de *status*, os apontadores de pilha e de quadro, entre outros. Durante uma troca de contexto, o conteúdo dos registradores

do processo atual é copiado para o PCB do processo e os registradores do processador são atualizados com os valores armazenados no PCB do novo processo.

Figura 3.5 – Bloco de Controle de Processo - PCB

Mais detalhadamente, as informações de **controle de processo** do PCB incluem:

- **Informação de estruturação do processo** –– a identificação dos processos filhos, ou a identificação de outros processos relacionados com o processo atual de alguma forma, que podem ser representados como uma fila, um anel ou outras estruturas de dados;
- **Informação de comunicação entre processos** –– *flags*, sinais e mensagens associadas à comunicação entre processos independentes;
- **Privilégios do processo** –– acesso permitido/não permitido aos recursos do sistema;
- **Informações de escalonamento** — informações de escalonamento de tempo de processador;
- **Informações de gerenciamento de memória** — tabela de páginas, limites de memória, tabela de segmentos;
- **Informações contábeis** — tempo total de processamento para execução do processo, limites de tempo, identificação de execução, etc.;
- **Informação de E/S** — lista de dispositivos E/S alocados ao processo, arquivos abertos, etc.

3.2.3 Criação de processos

No início da execução do sistema operacional, nenhum processo está ativo ou foi criado. Eles são criados normalmente a partir de um processo inicial com operações de **fork()** e **exec()** (veja a Seção 3.2.6), com a execução dos *scripts* iniciais do sistema operacional. Basicamente, existem quatro eventos principais fazem com que os processos sejam criados:

1. A partir do processo inicial do sistema (init ou systemd no Linux);

DRAFT

2. Execução de uma chamada de sistema de criação de processo por um processo em execução (processos criando outros processos automaticamente – /etc/init, /etc/crontab);
3. Solicitação de um usuário para criar um novo processo (interface gráfica ou linha de comando);
4. Início de uma tarefa em lote (sistemas em lote encontrados em computadores de grande porte).

Colocar um programa em execução, criando um novo processo, demanda que o sistema operacional realize diversas etapas, que elencamos a seguir:

1. **Atribuir um identificador único (PID)**: o PID (Identificador de Processo) é um número único atribuído a cada processo, usado para identificar e gerenciar o processo.
2. **Alocar uma entrada na tabela de processos**: a tabela de processos é uma estrutura de dados que armazena informações sobre todos os processos em execução no sistema. Antes de criar um novo processo, o sistema operacional aloca uma entrada na tabela de processos para armazenar as informações do novo processo.
3. **Reservar espaço de endereçamento para o processo**: antes de carregar o programa, o sistema operacional precisa alocar uma região de memória para o espaço de endereçamento do processo, que será usado para armazenar o código do programa, dados, pilha e outras estruturas necessárias para a execução do processo.
4. **Carregar o programa no espaço reservado**: uma vez que o espaço de endereçamento tenha sido reservado, o sistema operacional carrega o programa executável associado ao processo na memória principal.
5. **Colocar valores iniciais no PCB (Bloco de Controle de Processo)**: Antes de iniciar a execução do processo, o sistema operacional coloca valores iniciais no PCB para configurar o estado inicial do processo.
6. **Colocar o processo na fila apropriada**: O novo processo pode ser colocado na fila de processos prontos para execução, fila de processos aguardando E/S ou outras filas de gerenciamento de processos.
7. **Criar estruturas auxiliares**: Essas estruturas podem incluir tabelas de alocação de memória, tabelas de arquivos abertos, estruturas de sincronização e outras estruturas necessárias para suportar as operações do processo durante sua execução.

3.2.4 Estados de um processo

Um processo no sistema operacional, durante o seu ciclo de vida, passa normalmente por várias condições de execução, definidas como estados. O modelo de cinco estados de um processo é uma representação simplificada desses diferentes estados pelos quais passa um processo. Esses estados são geralmente definidos como:

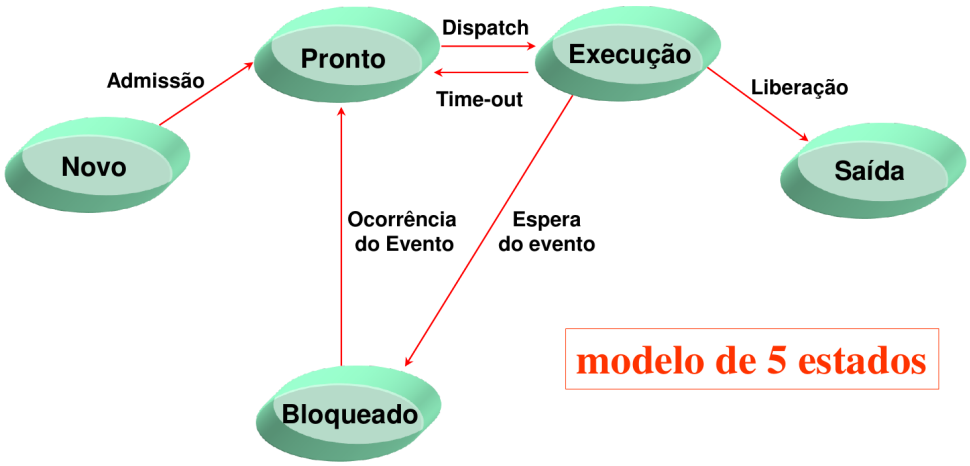
1. **Novo**: neste estado, o processo foi criado, mas ainda não começou a sua execução. O sistema operacional reservou os recursos necessários para o processo, como PID, espaço de endereçamento e PCB, mas o processo ainda não foi colocado na fila de execução.

DRAFT

2. **Pronto**: quando um processo está no estado pronto, significa que ele está pronto para ser executado pelo processador, mas está aguardando sua vez na fila de processos prontos.
3. **Execução**: quando um processo está no estado de execução, significa que ele está atualmente sendo executado pelo processador. Neste estado, o processo está utilizando ativamente os recursos do sistema, executando suas instruções e realizando operações de computação.
4. **Bloqueado ou em Espera**: se um processo está bloqueado, significa que ele está aguardando a conclusão de uma operação de E/S ou algum outro evento externo para continuar sua execução. Durante esse tempo, o processo não está utilizando o processador e não pode progredir até que a operação de E/S seja concluída ou o evento externo ocorra. O processo continua na memória principal, mas é colocado na fila de processos em estado de espera ou bloqueados.
5. **Encerrado ou em Saída**: quando um processo conclui sua execução, ele entra no estado encerrado. Isso pode ocorrer quando o programa termina sua execução normalmente ou quando é interrompido por algum motivo, como um erro de execução ou um comando de encerramento emitido pelo usuário. Nesse estado, o sistema operacional libera os recursos alocados para o processo e remove suas entradas associadas da tabela de processos.

DRAFT

Figura 3.6 – Estados do Processo - Modelo Cinco Estados



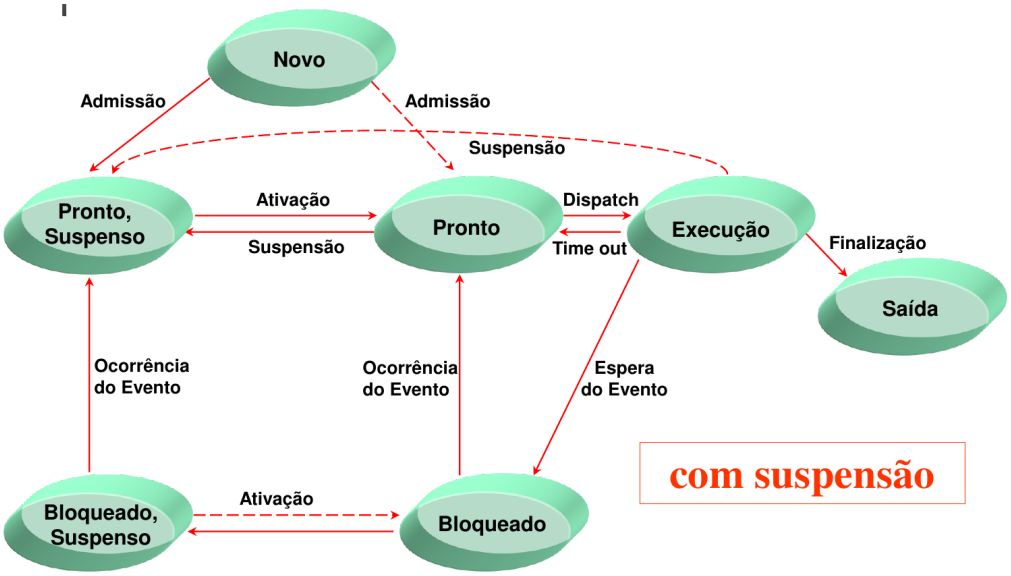
Esses cinco estados fornecem uma maneira simplificada de entender e gerenciar a execução de processos em um sistema operacional, permitindo que o sistema faça a transição adequada entre os estados com base nas operações em andamento e nos eventos que ocorrem durante a execução do processo.

Alguns sistemas operacionais, por exemplo quando a memória está muito cheia, podem retirar alguns processos da memória principal e enviá-los para a memória secundária. Quando um processo que está no estado **pronto** é enviado para a memória secundária, ele passa para

o estado **pronto suspenso**.

Uma vez que a memória principal tenha espaço suficiente para o processo, o processo será trazido de volta para a memória principal e estará no estado **pronto**. Sempre que o processo que está no estado **bloqueado** na memória principal e for enviado para a memória secundária, o estado do processo é alterado para **bloqueado suspenso**.

Figura 3.7 – Estados do Processo - Modelo com Suspensão



Um modelo dos estados de um processo, que inclui a suspensão de processos, por ser visto na Figura 3.7. Alguns dos motivos para a suspensão de um processo são os seguintes:

- ***Swapping***: O sistema operacional precisa liberar memória principal suficiente para trazer um processo que está pronto para executar;
- **Uso excessivo de recursos**: O sistema operacional suspeita que o processo está com uso demasiado de recursos;
- **Solicitação interativa do usuário**: por exemplo a depuração ou em conexão com o uso de um recurso;
- **Temporização**: Um processo pode ser executado periodicamente (por exemplo, um processo de contabilidade ou de monitoramento do sistema) e pode ser suspenso enquanto espera pela próxima execução;
- **Solicitação do Processo Pai**: Um processo pai pode suspender a execução de um descendente para examinar ou modificar o processo suspenso, ou para coordenar a atividade de vários descendentes.

3.2.5 Controle de processos

A troca de contexto é um procedimento onde o processador suspende a execução de um processo e passa a executar um novo processo. É um procedimento essencial em sistemas operacionais multitarefa preemptivos, nos quais múltiplos processos competem pelo tempo de processador. Aqui estão alguns motivos para a troca de contexto de um processo:

DRA

1. **Fatia de tempo (*time slice*) expirada**: cada processo recebe uma fatia de tempo de processador, chamada de *quantum*. Ao término deste tempo, o processo é pausado e outro processo na fila de prontos é selecionado para ser executado. Isso garante que todos os processos recebam tempo de processamento e nenhum deles monopolize o processador.
2. **Interrupção**: se o processador receber uma interrupção, deve suspender a execução do processo atual, desviar para o sistema operacional e realizar o tratamento da interrupção. Ao final deste atendimento, pode retomar a execução do mesmo ou de um outro processo, de acordo com a prioridade da fila de prontos.
3. **Chamada de sistema**: um processo faz uma chamada de sistema quando precisa executar uma operação que requer a intervenção do sistema operacional (por exemplo, realizar uma operação de E/S, solicitar a alocação de memória, etc.). O sistema operacional pode interromper a execução do processo atual e lidar com a solicitação, enquanto outro processo é selecionado para execução.
4. **Ocorrência de exceção:** quando o processo gera uma exceção durante sua execução, tais como acesso a uma página não existente (*page fault*, *breakpoint*, entre outras.
5. **Término da execução**: quando o processo termina a sua execução, seja normalmente ou por ocorrência de algum erro de execução.

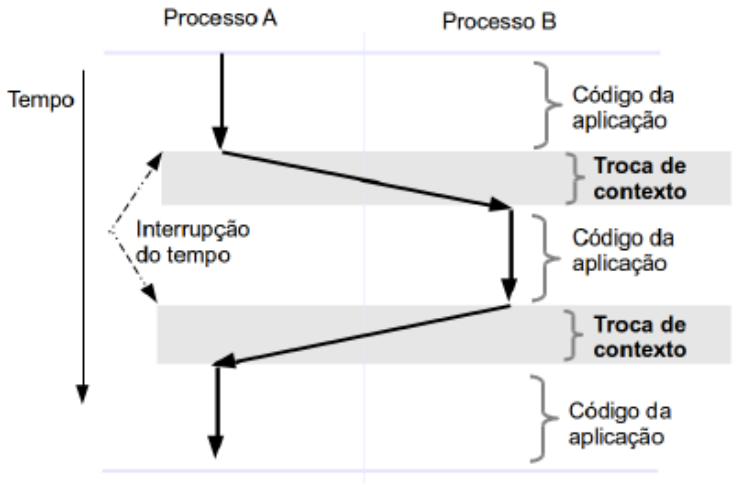
O processo de troca de contexto é trabalhoso e requer diversos passos que são executados pelo sistema operacional:

1. **Salvar o estado do processo atual**: o sistema operacional salva o estado do processo atualmente em execução, incluindo os valores dos registradores do processador, apontador de instruções, apontador de pilha e outras informações relevantes. Isso geralmente é feito em uma estrutura de dados chamada de Bloco de Controle de Processo (PCB);
2. **Mudar o estado do processo**: mudar o estado do processo no PCB para bloqueado e mudar o processo para a fila apropriada (pronto $\rightarrow$ bloqueado);
3. **Selecionar um novo processo**: o sistema operacional decide qual processo será executado em seguida, geralmente com base em um algoritmo de escalonamento;
4. **Restaurar o estado do novo processo**: o sistema operacional carrega o estado salvo do próximo processo a ser executado, incluindo os valores dos registradores do processador, apontador de instruções, apontador de pilha e outras informações relevantes, a partir do PCB;
5. **Mudar o estado do novo processo**: atualizar o estado do novo processo no PCB, atualizar a tabela de página e mudar a fila de execução (bloqueado $\rightarrow$ pronto);
6. **Continuar a execução**: o sistema operacional transfere o controle para o novo processo, permitindo que ele continue a execução a partir do ponto em que foi interrompido anteriormente.

Este ciclo de troca de contexto ocorre continuamente em sistemas operacionais multitarefa preemptivos para permitir que múltiplos processos compartilhem eficientemente os recursos do sistema.

DRAFT

Figura 3.8 – Troca de Contexto



Os processos, como tudo na vida, tem início e fim. Há várias razões pelas quais um processo pode ser terminado em um sistema operacional. Aqui estão algumas das mais comuns:

1. **Conclusão da tarefa**: o processo pode ter concluído sua tarefa com sucesso e terminado naturalmente;
2. **Erro de execução**: o processo pode encontrar um erro durante sua execução, levando-o a terminar prematuramente. Isso pode incluir tais como: erros de segmentação; violações de acesso à memória; instrução ilegal; divisão por zero; falta de recursos; entre outros.
3. **Término forçado**:
   - **Interrupção do usuário**: um usuário pode interromper manualmente a execução do processo através de um comando específico, como o comando 'kill' no Unix/Linux;
   - **Morte do processo pai**: um processo pode ser terminado caso o seu processo pai também seja terminado.
   - **Consumo de recursos**: o sistema operacional pode encerrar um processo se ele estiver consumindo muitos recursos (processador, memória, etc.) e estiver impactando negativamente o desempenho do sistema como um todo.
   - **Limite de tempo de execução**: Um processo pode ser programaticamente encerrado pelo sistema operacional em resposta a determinadas condições, como um tempo limite de execução.

Essas são apenas algumas das razões comuns para o término de um processo em um sistema operacional, sendo que as circunstâncias específicas podem variar dependendo do sistema operacional e do ambiente de execução.

DRAFT

3.2.6 Fork vs Exec

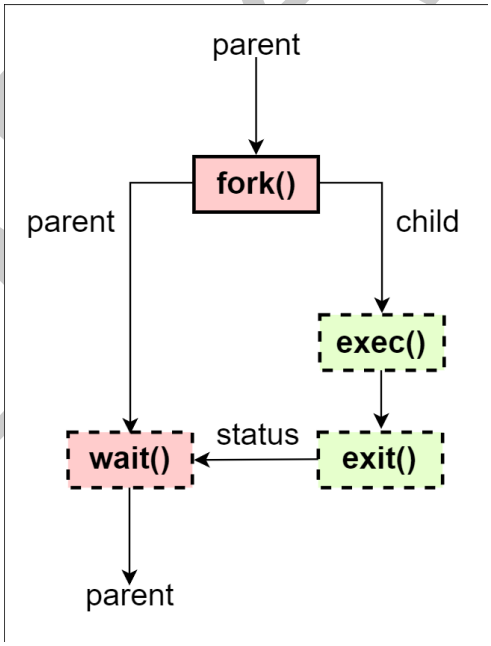
Os sistemas operacionais do tipo POSIX são aqueles que seguem as especificações do POSIX (Portable Operating System Interface), um conjunto de normas técnicas definido pelo IEEE e baseado no sistema operacional UNIX. O objetivo do POSIX é garantir compatibilidade entre diferentes sistemas operacionais, permitindo que aplicações desenvolvidas para um sistema compatível possam ser executadas em outros sistemas com poucas ou nenhuma modificação.

Nos sistemas operacionais desse tipo, um processo pode gerar cópias, chamadas de processos **filhos**, que compartilham o mesmo código dos processos pais, contudo **não compartilham** o mesmo espaço de endereçamento.

As chamadas de sistema **fork()** e **exec()** são usadas nos sistemas POSIX para criar novos processos e executar programas. Enquanto o **fork()** é usado para criar um novo processo que é uma cópia exata do processo pai, o **exec()** é usado para substituir o conteúdo do processo atual por um novo programa.

Na realidade a criação de um novo processo envolve uma combinação de chamadas **fork()** e **exec()**. Ou seja, o processo pai faz primeiro a chamada **fork()** e o processo filho, em seguida, faz a chamada **exec()**, criando assim um novo processo, que mantém ainda a relação de filho com o processo pai original.

Figura 3.9 – Criação de um novo processo



Aqui estão as principais diferenças entre essas duas chamadas:

- **fork()**:
  - A chamada de sistema **fork()** é usada para criar um novo processo que é uma cópia exata do processo pai;
  - Após a chamada **fork()**, dois processos são executados: o processo pai (processo original que fez a chamada) e o processo filho (cópia do processo pai);

  - O processo filho herda uma cópia do espaço de endereço do processo pai, incluindo variáveis, descritores de arquivo e outros recursos;
  - Contudo, o filho não herda as travas de memória ou semáforos do processo pai, nem as operações de E/S assíncronas pendentes do seu pai, nem herda nenhum contexto de E/S assíncrono do seu pai;
  - A função **fork()** retorna um valor inteiro para ambos os processos: para o processo pai, retorna o **pid** (identificador de processo) do processo filho; para o processo filho, retorna 0.
  - O processo filho geralmente é usado para executar uma tarefa diferente do processo pai, embora possa compartilhar recursos comuns.
- **exec()**:
  - A chamada de sistema **exec()** é usada para substituir o conteúdo do processo atual por um novo programa;
  - Ela carrega um novo programa na memória e começa a executá-lo, descartando o programa anterior;
  - Existem várias variantes da função **exec()** (por exemplo, **execl(), execv(), execle(**, etc.), cada uma com diferentes formas de passar argumentos para o novo programa;
  - A função **exec()** não cria um novo processo; em vez disso, ela substitui o código e o espaço de endereço do processo atual pelo programa especificado na chamada da função;
  - Se a chamada **exec()** for bem-sucedida, o processo continuará sua execução com o novo programa. Se falhar, o processo manterá seu estado anterior.

Tanto o processo pai como o filho são executados simultaneamente no caso do **fork()**, enquanto que o controle nunca retorna ao programa original caso a chamada **exec()** seja executada sem erro. No caso da chamada **fork()** o processo filho ganha um PID novo, enquanto que no caso do **exec()** o processo filho recebe o mesmo PID do processo pai. O programa a seguir procura exemplificar o uso dessas duas funções.

```
#include <stdio.h>
#include <unistd.h>
int main () {
    if (fork() == 0){
        // O processo filho recebe 0 como retorno
        printf("Alô do processo filho %d!\n" , getpid());
        execl ("./teste", "teste", 0); /* troca programa */
    }
    else
    // O processo pai recebe um valor diferente de 0 como retorno
        printf("Alô do processo pai %d!\n", getpid());
        wait();
return 0;
```

DRAFT

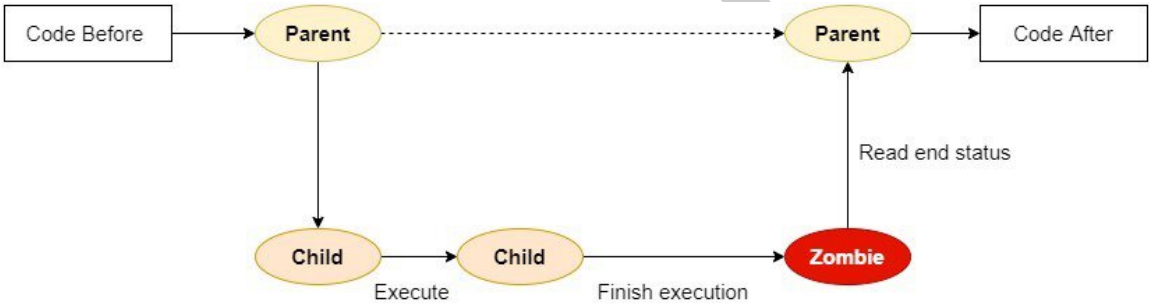
```
}

#include <stdio.h>
#include <unistd.h>
int main()   {
    printf("Alô do processo teste %d!\n" , getpid());
    return 0;
}
```

Processos Zumbi

Geralmente, quando um processo filho conclui sua execução e termina, é responsabilidade do processo pai remover o processo filho da tabela de processos. No entanto, em algumas circunstâncias, isso pode não ocorrer conforme o esperado, resultando na criação de um processo **zumbi**, também conhecido como processo **defunto** ou **morto**.

Figura 3.10 – Processo Zumbi



https://www.baeldung.com/cs/process-lifecycle-zombie-state

Em termos simples, um processo zumbi é aquele que está morto, mas permanece presente na tabela de processos do sistema. Idealmente, ele deveria ter sido removido da tabela de processos assim que concluísse sua execução. No entanto, por algum motivo, o processo pai não o removeu corretamente após a execução. Isso ocorre porque, quando o processo termina, o núcleo não pode apagar imediatamente sua entrada: ele precisa esperar o pai coletar o *status* de término.

A condição de zumbi ocorre quando o processo pai não consegue receber o *status* de término do processo filho, que é necessário para remover o processo da memória. Como resultado, daí o seu nome, o processo zumbi permanece na tabela de processos, consumindo recursos do sistema, como espaço de memória. Isso pode ocorrer por várias causas, tais como:

- O pai não chamou **wait()** ou **waitpid()**: Esse é o motivo clássico. Se o processo pai nunca executa nenhuma chamada de espera, a entrada do filho (PID + exit status) não pode ser removida pelo núcleo (*kernel*).
- O pai terminou antes do filho (mas ainda não foi adotado pelo init/systemd). Se o pai morre antes do filho, inicialmente o filho fica sem pai. Depois, ele será adotado pelo init

DRAFT

(PID 1) ou systemd, que fará o *reaping* (coletará o *status*). Mas antes dessa adoção, ele pode aparecer como zumbi por pouco tempo.

- O pai deliberadamente ignora sinais SIGCHLD. Um processo pai pode ignorar o sinal SIGCHLD, ou tratá-lo incorretamente, de modo que não chama **wait()** no handler.
- O pai está ocupado em um loop infinito (ou bloqueado). Se o processo pai: entra em um loop sem checar filhos, está bloqueado em I/O, ou fica esperando outra coisa e nunca chega ao ponto do código onde chamaria **wait()**, então os filhos finalizados permanecem como zumbis. Isso impede que filhos terminados sejam limpos.
- O pai criou muitos filhos rapidamente e não conseguiu acompanhar. Casos de processos que criam centenas ou milhares de filhos em sequência: o pai não chama **wait()** rápido o suficiente, e filhos terminam antes que o pai processe seus sinais. Isso pode causar acúmulo temporário (ou permanente) de zumbis.

Note que os processos zumbis geralmente não representam um problema sério por si só, pois ocupam uma quantidade mínima de recursos. No entanto, se muitos processos zumbis forem se acumulando, eles podem eventualmente esgotar os recursos do sistema e afetar o desempenho geral.

3.3 ESCALONAMENTO DE PROCESSOS

O escalonamento de processos em um sistema operacional permite a execução de múltiplos processos de forma eficiente em um ambiente compartilhado de recursos computacionais, como processador e memória. Simplificadamente, consiste em decidir qual processo será executado em seguida e por quanto tempo, entre os diversos processos prontos para serem executados. As estratégias de escalonamento adotadas pelo sistema operacional têm por objetivo:

- Maximizar a produtividade (*throughput*), processando o maior número de tarefas por unidade de tempo;
- Minimizar o tempo de resposta para usuários interativos, priorizando as tarefas interativas para garantir uma experiência fluida para o usuário;
- Minimizar o tempo de espera, reduzindo o tempo que um processo fica aguardando para ser executado;
- Minimizar o tempo de retorno (*turnaround*), reduzindo o tempo total que um processo leva para ser concluído, desde sua submissão até sua finalização. É a soma do tempo de espera para ser carregado na memória, tempo de espera na fila dos prontos, tempo em execução no processador e o tempo de espera por recursos;
- Privilegiar a execução de tarefas críticas, garantido o seu atendimento em detrimento de outros processos de menor prioridade;
- Manter o processador ocupado a maior parte do tempo, executando tarefas pendentes e diminuindo o seu tempo ocioso;
- Ser previsível, assegurando que o tempo de execução de cada processo seja relativa-

DRAFT

mente consistente e previsível;

- Manter a justiça, garantindo que todos os processos recebam uma parcela justa do tempo de processador.

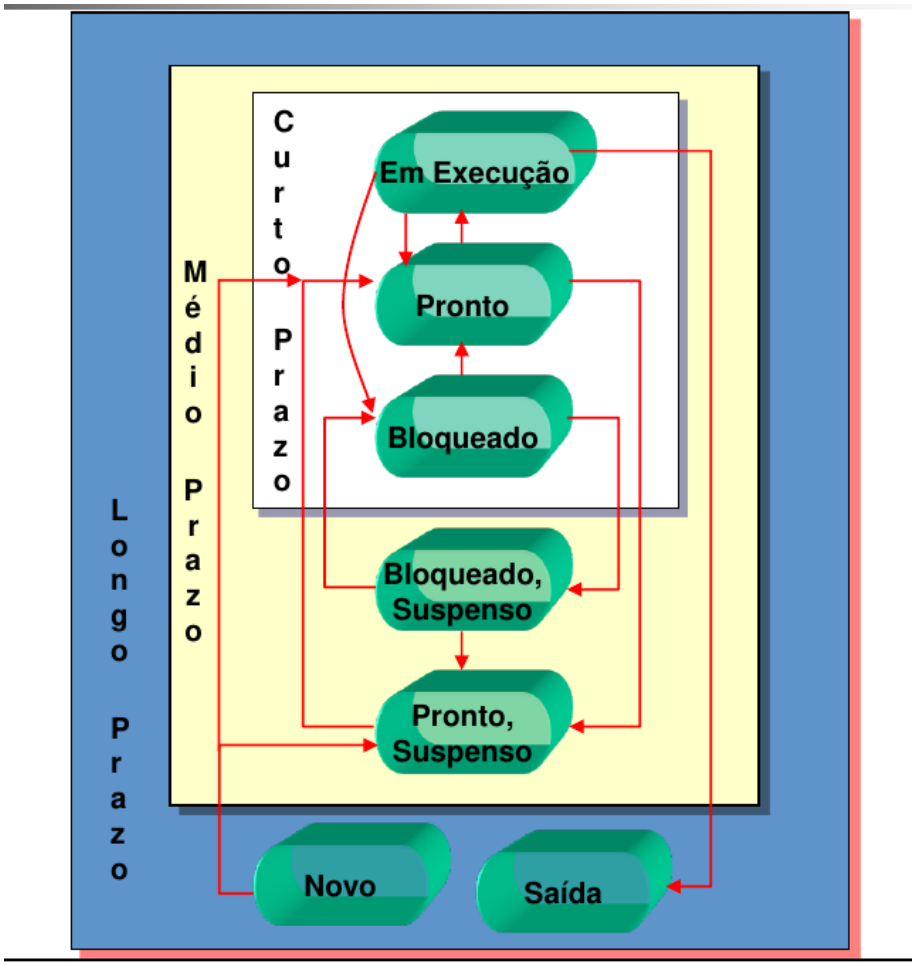
Esse objetivos podem ser divergentes em algumas situações, obrigando que sejam feitas escolhas, de acordo com a carga de trabalho que o sistema operacional deve executar. Existem diversos níveis de escalonamento em um sistema operacional, a saber:

- **Escalonamento de Longo Prazo**: é responsável por decidir os novos processos que serão admitidos na memória principal para execução cuida da admissão de novos processos no sistema.
  - Quando um processo é criado, ele entra na fila de processos de entrada, onde aguarda a sua admissão na memória principal;
  - O escalonador de longo prazo decide quais processos devem ser carregados na memória principal, com base em critérios como a disponibilidade de recursos de memória e o estado atual do sistema;
  - Este tipo de escalonamento ajuda a controlar a taxa de aceitação de novos processos no sistema, evitando sobrecarga de memória e garantindo que a quantidade de processos na memória seja gerenciável.
  - O escalonamento de longo prazo procura manter uma mistura de processos tipo *CPU-bound* (com mais uso de processador) e *I/O bound* (com mais operações de E/S) ativos no sistema.

  Se mais processos são aceitos é menos provável que todos os processos estejam bloqueados em um determinado instante de tempo, aumentando assim a concorrência (melhor uso do processador). Cada processo terá uma fração menor do tempo de uso do processador. Se o número de processos ativos for muito alto, a sobrecarga de trabalho causada pela troca de processos será tão grande que o tempo útil de utilização do processador cairá.
- **Escalonamento de Médio Prazo**: cuida da admissão de processos suspensos (completamente fora da memória principal) e implementa a troca (*swap*) dos processos da memória para a memória secundária, como o disco rígido, e vice-versa.
  - Quando há escassez de memória principal, alguns processos podem ser movidos para o armazenamento secundário para liberar espaço para outros processos mais urgentes;
  - Este tipo de escalonamento ajuda a otimizar a utilização dos recursos de memória, garantindo que os processos mais importantes tenham acesso rápido à memória principal quando necessário.
- **Escalonamento de Curto Prazo**: cuida da seleção de processos que estão na condição de prontos para serem executados pelo processador.
  - O escalonador de curto prazo decide qual processo na memória principal será executado a seguir e por quanto tempo;
  - Este tipo de escalonamento é geralmente realizado de forma preemptiva, o que

DRAFT

Figura 3.11 – Escalonamento de Processos



significa que um processo pode ser interrompido para permitir a execução de outro processo mais prioritário.

– Os algoritmos de escalonamento de curto prazo são responsáveis por decidir a ordem de execução dos processos no processador, com base em critérios como prioridade, tempo de espera, fatia de tempo, etc.

Em síntese, o escalonamento de longo prazo, médio prazo e curto prazo trabalham juntos para garantir que os recursos do sistema, como memória e processador, sejam utilizados de forma eficiente, maximizando o desempenho e a capacidade de resposta do sistema operacional.

3.3.1 Escalonamento de curto prazo

O escalonador determina qual será o próximo processo a ser executado e é acionado pelos seguintes eventos que podem ocorrer durante a execução do sistema operacional:

- Interrupção de relógio;
- Interrupção de E/S;
- Chamadas ao sistema;
- Recepção de sinais.

A avaliação de uma política de escalonamento de um sistema operacional envolve diversos parâmetros que ajudam a determinar a eficiência e a eficácia do escalonamento de processos. Em seguida relacionamos os principais parâmetros de avaliação:

1. **Tempo de *Turnaround***: O tempo total que um processo leva desde o momento em que é submetido até o momento em que é concluído. Um menor tempo de *turnaround* indica que os processos estão sendo concluídos rapidamente, melhorando a resposta geral do sistema.
2. **Tempo de Resposta**: o tempo desde a submissão de um processo até a primeira resposta produzida. É um parâmetro crítico em sistemas interativos onde é importante que os usuários obtenham uma resposta rápida às suas entradas.
3. **Tempo de Espera**: o tempo total que um processo passa na fila de espera antes de ser executado. Quanto menor for o tempo de espera indica que os processos estão sendo atendidos de forma mais eficiente.
4. **Utilização do processador**: A porcentagem do tempo em que o processador está ativo executando processos. Uma maior utilização do processador indica que os recursos estão sendo usados de forma eficiente.
5. **Vazão (Throughput)**: O número de processos que são concluídos em um determinado período de tempo. Quanto maior for a taxa de transferência, indica que o sistema está processando um grande número de processos de forma eficiente.
6. **Justiça**: uma medida de que os processos recebem uma quantidade equitativa de tempo de processador. Isso é importante para garantir que todos os processos, especialmente em um ambiente multiusuário, tenham uma chance justa de serem executados.
7. **Equilíbrio de Carga**: a distribuição equitativa da carga de trabalho entre todos os processadores disponíveis. Ajuda a prevenir sobrecarga de alguns processadores, enquanto outros estão subutilizados, melhorando a eficiência geral do sistema.
8. **Previsibilidade**: a constância do desempenho do sistema ao longo do tempo. Sistemas previsíveis são mais confiáveis e mais fáceis de gerenciar e otimizar.
9. **Sobrecarga de Escalonamento**: o tempo e os recursos gastos pelo sistema operacional para realizar as operações de escalonamento. Quanto menor for esse valor, mais tempo será gasto na execução real dos processos.
10. **Tratamento de Prioridades**: a eficácia com que a política de escalonamento trata processos com diferentes prioridades. Isso é muito importante, por exemplo, em sistemas de tempo real.

Esses parâmetros são usados para avaliar e comparar diferentes políticas de escalonamento, ajudando a determinar qual política é mais adequada para um determinado ambiente ou carga de trabalho. As políticas de escalonamento podem ser classificadas segundo a possibilidade de o sistema operacional interromper ou não um processo em execução e substituí-lo por outro.

- **Escalonamento Não-Preemptivo**: Foi o primeiro tipo de escalonamento implementado nos sistema multiprogramáveis, onde predominava o processamento em lote (batch).

DRAFT

O processo fica executando até terminar ou até ser bloqueado em consequência a uma chamada ao sistema (E/S ou pedido de recurso do S.O.);

- **Escalonamento Preemptivo**: caracterizado pela possibilidade de o sistema operacional suspender a execução de um processo e substituí-lo por outro que esteja no estado de pronto. O processo em execução pode ser interrompido:
  - Quando chega um novo processo;
  - Se um outro processo de maior prioridade fica pronto;
  - Quando interrompido pelo relógio (atingiu a fatia de tempo determinada – *timeslice* ou *quantum*);

  Isso evita que um processo monopolize o processador, possibilitando um melhor funcionamento do sistema operacional como um todo.

As estratégias de curto prazo determinam a ordem na qual os processos em estado pronto serão executados pelo processador. Cada algoritmo possui características e objetivos específicos, influenciando diretamente no desempenho do sistema. Alguns dos algoritmos mais comuns incluem:

1. **Por Prioridade**: neste algoritmo, cada processo recebe uma prioridade, e o escalonador escolhe o processo com a maior prioridade para execução. Se houver processos com a mesma prioridade, pode ser aplicado outro critério, como o FCFS. Esse método é flexível, permite privilegiar processos críticos ou sensíveis ao tempo, mas pode levar à inanição de processos com prioridades baixas caso haja muitos processos com prioridades altas. Pode ser preemptivo (permite a interrupção da execução de um processo por outro de maior prioridade) ou não preemptivo.
2. **First-Come First-Served (FCFS)**: é um dos algoritmos mais simples. Os processos são executados na ordem em que chegaram à fila de prontos. Ele é fácil de entender e implementar, mas pode resultar em tempos de espera longos, especialmente quando processos longos que chegam primeiro. Um exemplo prático é o atendimento em filas como bancos e caixas de lojas.
3. **Round Robin**: cada processo é atribuído um pequeno intervalo de tempo de processador, chamado de fatia de tempo ou *quantum*. Os processos são executados em ordem circular e, quando o *quantum* de um processo termina, ele é colocado no final da fila de prontos. Isso garante um bom tempo de resposta para processos interativos, evita a estagnação e garante justiça entre os processos, mas se o quantum de cada processo for muito curto, pode gerar maior sobrecarga (*overhead*) de escalonamento devido à troca frequente de contexto, e se o *quantum* for muito longo, degenera em FCFS.
4. **Shortest Process Next (SPN)**: neste algoritmo, o processo com o menor tempo esperado de serviço é selecionado para execução. Isso minimiza o tempo médio de espera, favorecendo os processos mais curtos. Minimiza o tempo médio de espera e o tempo médio de resposta, no entanto, pode causar inanição para processos longos, pois eles podem nunca serem executados se houver muitos processos curtos na fila. Outra desavantagem é que requer uma estimativa precisa do tempo de execução, o que pode ser

DRAFT

difícil em alguns casos. É muito utilizado em sistemas em lote (batch).

5. **Shortest Remaining Time (SRT)**: similar ao SPN, o SRT seleciona o próximo processo com o menor tempo de execução restante. A diferença é que, se um novo processo chegar com um tempo de execução menor do que o tempo restante do processo em execução, o processo em execução é interrompido e colocado de volta na fila de prontos. Minimiza o tempo médio de espera e o tempo médio de resposta adaptativamente. Isso garante que os processos mais curtos sejam atendidos o mais rápido possível, mas pode resultar em uma grande quantidade de trocas de contexto, e sobrecarga (*overhead*), adicional pela necessidade de atualização constante do tempo restante de cada processo.
6. **Highest Response Ratio Next (HRRN)**: este algoritmo calcula uma relação de resposta para cada processo, levando em consideração tanto o tempo de espera quanto o tempo de execução. O processo com a maior relação de resposta é selecionado para execução. Equilibra o tempo de espera e o tempo de execução, favorecendo processos que já esperam há mais tempo. Isso ajuda a evitar a inanição, pois leva em conta tanto o tempo de espera quanto o tempo de execução. Mas o cálculo da razão HRRN pode gerar *overhead* adicional para o algoritmo de escalonamento.
7. **Feedback**: o algoritmo de *feedback* é um tipo de escalonamento de múltiplas filas, onde os processos são colocados em várias filas de acordo com sua prioridade. Os processos são inicialmente colocados na fila de maior prioridade e, se não forem concluídos durante o quantum de tempo atribuído, são movidos para a próxima fila de prioridade mais baixa. Processos que utilizam mais processador são colocados em filas com menor prioridade, enquanto processos que utilizam menos processador são promovidos para filas com maior prioridade. Em resumo, baseia-se no histórico de execução dos processos para ajustar suas prioridades. Adapta-se ao comportamento dos processos, priorizando aqueles que mais necessitam de recursos. Mas tem uma implementação mais complexa e exige mecanismos para monitorar o histórico de execução.

A escolha do algoritmo ideal depende dos objetivos do sistema, das características dos processos e das cargas de trabalho. Cada algoritmo possui vantagens e desvantagens que devem ser cuidadosamente ponderadas. Fatores como tempo médio de espera, tempo médio de resposta, *throughput* e utilização do processador também influenciam na escolha do algoritmo.

3.4 *THREADS*

3.4.1 Definição

As *threads* podem ser descritas como fluxos de execução independentes que operam dentro de um mesmo processo, sendo suportadas pela maioria dos sistemas operacionais modernos (SILBERSCHATZ; GALVIN; GAGNE, 2013).

As *threads* de um processo possuem seu próprio contexto de execução, que inclui o identificador único da *thread* (tid), o estado da pilha de execução, além dos valores do apontador

DRAFT

de instrução e dos registradores de uso geral do processador. No entanto, elas compartilham o mesmo espaço de endereçamento de memória, os arquivos abertos e outros recursos que compõem o contexto global do processo, como ilustrado na Figura 3.12.

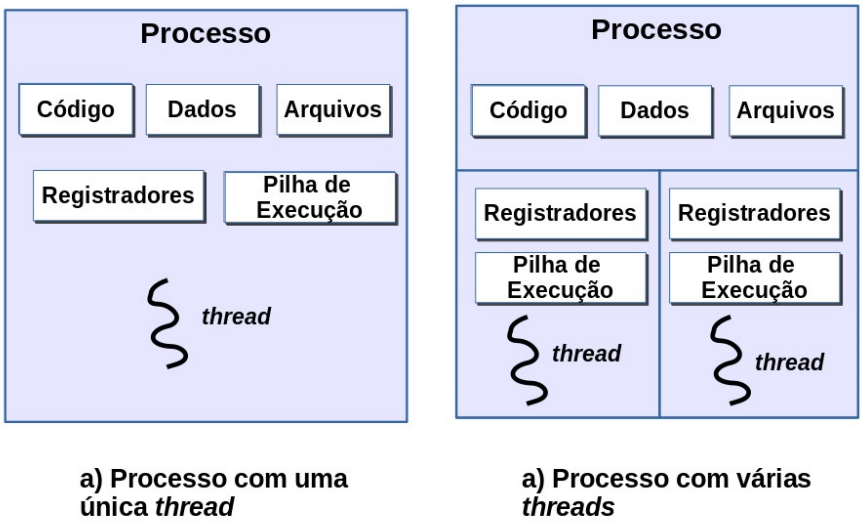


Figura 3.12 – Processos com uma única *thread* e com várias *threads*

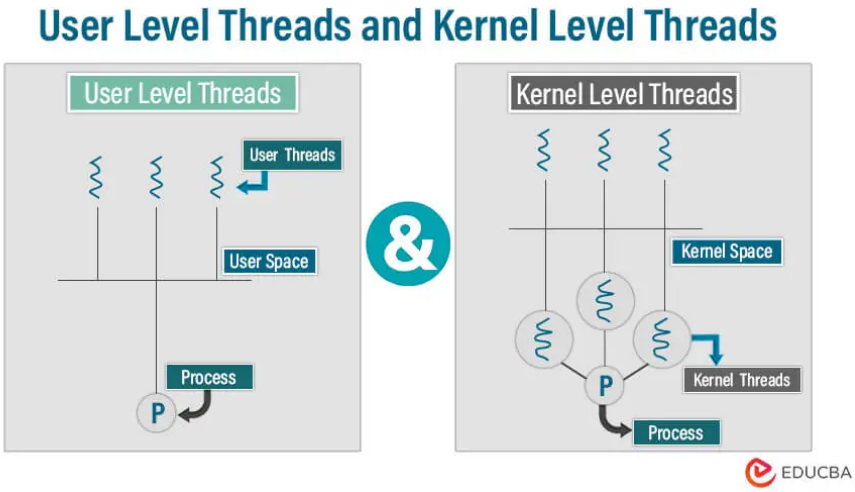
Todo processo começa a executar a partir de uma *thread* principal que é responsável pela criação das outras *threads*. Quando uma nova *thread* é criada, ela executa de forma concorrente ou paralela com as demais *threads* do processo. Entretanto, a execução de processos e *threads* é diferente nos seguintes pontos:

- As *threads* **não** são organizadas de forma hierárquica, como no caso de processos (processos pai e processos filhos). Todas as *threads* associadas a um processo formam um grupo de *threads* de mesmo nível, independente da ordem de criação ou de quais *threads* foram criadas por quais outras *threads*.

- A *thread* principal distingue-se das demais apenas por ser sempre a primeira *thread* a ser executada e, dependendo da implementação da biblioteca de gerência de *threads* usada, o processo encerra automaticamente quando a *thread* principal termina.

O contexto específico de uma *thread* é semelhante ao contexto de uma função e, consequentemente, a troca de contexto entre *threads* implica no salvamento e recuperação de contextos relativamente leves, a exemplo do que ocorre numa chamada de função dentro de um programa. Aplicações que utilizam mais de uma *thread* representam uma alternativa usual à programação paralela baseada em múltiplos processos. Isso ocorre porque a troca de

Figura 3.13 – Threads de Usuário e Kernel



contexto entre *threads* é mais simples e rápida do que entre processos. Além disso, por compartilharem o mesmo espaço de endereçamento, a comunicação entre *threads* é facilitada e pode ser realizada diretamente por meio da leitura e escrita de variáveis na memória. Isso elimina a necessidade do uso de mecanismos especiais, mais restritos e, usualmente, mais lentos de comunicação entre processos oferecidos pelos sistemas operacionais (veja a Seção 4.1).

3.4.2 *Threads* de usuário e de kernel

As *threads* podem ser gerenciadas pelo sistema operacional, quando são chamadas de *threads de kernel* ou por uma biblioteca do usuário, sem a intervenção direta do sistema operacional, quando são chamadas de *threads* de usuário. Em alguns sistemas, *threads* de usuário são chamadas simplesmente de *threads* e *threads* de *kernel* são referidas como processos leves (LWP - *lightweight process*).

Quando as *threads* são gerenciadas pelo sistema operacional, as trocas de contexto das *threads* são executadas sem o conhecimento do aplicativo. Da mesma forma, quando as *threads* são implementadas no espaço de usuário, as trocas de contexto das *threads* são executadas por uma biblioteca de aplicativos, sem o conhecimento do sistema operacional. Há também abordagens híbridas nas quais, normalmente, uma biblioteca de espaço de usuário vincula uma ou mais *threads* de usuário a uma ou mais *threads* de *kernel* (veja a Seção 3.5) (TANENBAUM; WOODHULL, 2006) .

As *threads* de usuário tendem a ser mais portáteis e realizar trocas de contexto de forma mais rápida e econômica em comparação com as *threads* de *kernel*. No entanto, as *threads* de *kernel* têm a vantagem de tirar proveito de máquinas com múltiplos processadores e não enfrentam problemas de bloqueio de outras *threads* do mesmo processo durante operações de E/S.

DRAFT

Threads de kernel

As *threads* de *kernel* são suportadas e gerenciadas diretamente pelo sistema operacional. Uma *thread* do *kernel* é a entidade escalonável, ou seja, o escalonador do sistema conhece e gerencia todas as *threads* do *kernel*. Existe um bloco de controle de processo (PCB - do inglês *Process Control Block*) por processo e um bloco de controle de *thread* (TCB - do inglês *Thread Control Block*) por *thread* no sistema operacional. Uma *thread* de *kernel* executa dentro de um processo, mas pode ser referenciada por uma outra *thread* no sistema operacional.

As *threads* de *kernel* possuem vantagens como o sistema operacional possuir pleno conhecimento de todas as *threads*, o que facilita a coordenação e o gerenciamento eficiente do sistema. Além disso, o escalonador pode tomar decisões mais informadas, como priorizar o tempo de processador para processos com um grande número de *threads*, otimizando assim o desempenho do sistema. Esse tipo de *thread* também se destaca em aplicações que frequentemente realizam operações de E/S que podem bloquear a *thread*, pois o sistema operacional está mais apto a lidar com essas situações de forma eficaz.

Entretanto, apresentam como desvantagem o fato de que o sistema operacional é responsável por gerenciar e escalonar todas as *threads*, o que pode resultar em uma sobrecarga significativa e aumentar a complexidade do sistema. Além disso, uma mudança do modo de execução do processador, de normal para privilegiado, é necessária para transferir o controle de uma *thread* para outra dentro de um processo, o que pode impactar negativamente o desempenho. Outro ponto é que as *threads* de *kernel* tendem a ser mais lentas para serem criadas e gerenciadas em comparação com as *threads* de usuário.

Threads de usuário

Uma *thread* de usuário é uma entidade usada por programadores para lidar com múltiplos fluxos de controle dentro de um programa. Uma *thread* de usuário existe apenas dentro de um processo, e as *threads* em um processo não podem referenciar *threads* em outro processo.

As *threads* de usuário possuem como vantagem o fato de poderem ser implementadas em sistemas operacionais que não suportam *threads* de *kernel*, o que aumenta sua acessibilidade e aplicabilidade. Além disso, não requerem modificações no sistema operacional, facilitando sua integração em ambientes variados. Sua representação é simples, com elementos como apontador de instruções, registradores e pilha armazenados no espaço de endereçamento do processo em nível de usuário, resultando em uma gestão simplificada. A criação, troca e sincronização de *threads* podem ser realizadas em nível de usuário sem intervenção do sistema operacional, tornando o gerenciamento mais flexível e ágil. Além disso, são conhecidas por sua eficiência, visto que o custo da troca de contexto entre *threads* é comparável ao de uma simples chamada de função.

Entre as desvantagens das *threads* de usuário podemos elencar a falta de coordenação entre o gerenciador de *threads* em nível de usuário e do sistema operacional. Isso pode resultar

DRAFT

no bloqueio de um processo quando uma de suas *threads* fica bloqueada, mesmo que outras *threads* estejam disponíveis para execução. Além disso, o processo recebe a mesma alocação de tempo de processador, independentemente do número de *threads* que possui. Outro problema é que, quando uma *thread* possui uma trava (*lock*), outras *threads* do mesmo processo podem ficar inativas. Para contornar esses problemas, pode ser necessário estabelecer alguma forma de comunicação entre o gerenciador de *threads* do sistema operacional e o de nível de usuário.

3.4.3 Modelos de Multithreading

Muitos sistemas operacionais suportam as *threads* de usuário e de *kernel* de uma forma combinada. Nesses esquemas, uma biblioteca de espaço de usuário vincula uma ou mais *threads* de usuário a uma ou mais *threads* de *kernel*. Existem basicamente três tipos de modelos *multithreading*.

- O primeiro é N:1, onde várias *threads* do espaço do usuário são executadas em uma única *thread* ou processo do sistema operacional. Isso tem a vantagem de ter uma troca de contexto muito rápida, mas não se beneficia de sistemas com vários núcleos, já que quando uma *thread* bloqueia, todas as demais *threads* de usuário também bloqueiam.

- Outro modelo possível é o 1:1, onde há um mapeamento de um para um entre *threads* de execução e *threads* do sistema operacional (threads de núcleo). Esse modelo aproveita todos os núcleos da máquina, mas a troca de contexto é lenta porque requer uma troca de contexto feita pelo sistema operacional.

- Outro esquema possível é M:N. Neste caso um número arbitrário de *threads* é mapeado em um número arbitrário de *threads* do sistema operacional. Com isso, você obtém trocas de contexto rápidas e aproveita todos os núcleos do seu computador. A principal desvantagem dessa abordagem é a complexidade que ela adiciona ao escalonador.

3.5 ESCALONAMENTO DE *THREADS*

O escalonamento de *threads* é o mecanismo responsável por decidir qual *thread* poderá utilizar o processador em cada momento. Isso pode ser feito diretamente pelo sistema operacional ou por uma biblioteca de execução em nível de usuário, dependendo da implementação de *threads* utilizada.

Quando uma biblioteca de execução dentro do próprio processo controla a criação, a troca de contexto e o escalonamento das *threads*, o núcleo do sistema operacional não toma conhe-

cimento direto da existência delas. A principal vantagem dessa abordagem é o baixo custo das operações de escalonamento, já que não é necessário realizar chamadas ao *kernel*. Entretanto, essa estratégia possui uma limitação importante: se uma *thread* realizar uma operação bloqueante, como uma leitura de disco ou uma chamada de entrada e saída que dependa do sistema operacional, todo o processo pode ficar bloqueado, impedindo a execução das demais *threads* daquele processo.

Quando são utilizadas *threads* em nível de *kernel*, cada *thread* é conhecida e gerenciada diretamente pelo sistema operacional. O escalonador do *kernel* decide qual *thread* será executada e pode realizar a troca de contexto entre *threads* pertencentes ao mesmo processo ou a processos diferentes. A vantagem desse modelo é que, caso uma *thread* fique bloqueada aguardando algum evento externo, o sistema operacional pode simplesmente selecionar outra *thread* pronta para execução, mantendo o processador ocupado. Essa abordagem oferece maior robustez e melhor integração com o restante do sistema, embora as operações de gerenciamento de *threads* geralmente tenham um custo um pouco maior, pois envolvem a participação direta do *kernel*.

Em sistemas modernos, é comum que um processo possua várias *threads* ativas simultaneamente, frequentemente em número muito superior ao de núcleos de processamento disponíveis. Diante disso, o escalonador precisa alternar rapidamente entre essas *threads*, atribuindo a cada uma pequenos intervalos de uso do processador. Essa alternância ocorre em intervalos muito curtos, o que cria para o usuário a impressão de que diversas *threads* estão sendo executadas ao mesmo tempo, mesmo quando há menos núcleos físicos do que tarefas concorrentes.

Assim como os processos, cada *thread* percorre diferentes estados ao longo de seu ciclo de vida. Quando uma *thread* possui todos os recursos necessários para executar, mas ainda não recebeu o direito de uso do processador, se encontra no **estado pronto**. Nesse estado, aguarda em uma fila aguardando ser escolhida pelo escalonador. Quando o escalonador decide atribuir o processador a essa *thread*, ela entra no **estado de execução**, no qual suas instruções passam a ser efetivamente processadas pelo processador. Em determinadas situações, no entanto, a *thread* pode precisar aguardar algum evento externo antes de continuar sua execução. Isso ocorre, por exemplo, quando ela solicita uma operação de leitura em disco, aguarda dados de entrada do usuário ou tenta adquirir um mecanismo de sincronização, como um *mutex*, que já está em posse de outra *thread*. Nessas circunstâncias, ela entra no estado bloqueado, também chamado de **estado de espera**, permanecendo nessa condição até que o evento aguardado ocorra. Após isso, a *thread* retorna ao estado pronto e volta a competir pelo uso do processador.

Enquanto as *threads* de *kernel* são geralmente escalonadas de forma preemptiva, em paralelo com os processos subjacentes, as *threads* de usuário têm a flexibilidade de utilizar tanto o escalonamento preemptivo quanto o não-preventivo.

No escalonamento preemptivo, o escalonador permite a execução de uma *thread* até que ocorra uma situação de bloqueio (geralmente uma chamada de função bloqueante) ou após decorrido o intervalo de tempo (*timeslice*) atribuído à *thread*. Em seguida, o escalonador retira o controle da *thread* sem nenhuma chance de oposição da *thread*. Isso geralmente é realizado

DRAFT

interrompendo a *thread* por meio de um sinal de interrupção de *hardware* (para *threads* de *kernel*) ou um sinal de interrupção de *software* (para *threads* de usuário).

No escalonamento não-preemptivo, após uma *thread* receber o controle do processador pelo escalonador, a execução da *thread* continua até que ocorra uma situação de bloqueio (novamente, uma chamada de função bloqueante que, em vez disso, retorna de volta ao escalonador), ou até que a *thread* devolva explicitamente o controle ao escalonador, de uma forma cooperativa.

Assim como no caso dos processos, diversos algoritmos podem ser utilizados para decidir qual *thread* pronta será executada em seguida. A escolha do algoritmo depende dos objetivos do sistema, que podem incluir a redução do tempo de resposta para aplicações interativas, o aumento da vazão de processamento ou a garantia de um uso justo de processamento entre diferentes tarefas. Um dos mecanismos mais simples e comuns é o escalonamento baseado em prioridades. Nesse modelo, cada *thread* recebe um valor numérico que representa sua prioridade relativa. O escalonador sempre seleciona para execução a *thread* com a maior prioridade entre aquelas que estão prontas. Contudo, isso pode levar a um problema conhecido como inanição (*starvation*). Esse fenômeno ocorre quando *threads* de baixa prioridade permanecem indefinidamente na fila de espera porque novas *threads* continuam chegando ao sistema. Uma técnica frequentemente utilizada para mitigar esse problema é chamada de envelhecimento (*aging*). Nela, a prioridade de uma *thread* aumenta gradualmente quanto mais tempo ela permanece aguardando na fila, garantindo que eventualmente ela tenha oportunidade de ser executada.

Outros algoritmos bastante utilizados são *Round-Robin*, amplamente utilizado em sistemas de tempo compartilhado; o escalonamento baseado em múltiplas filas com realimentação, conhecido como Multilevel Feedback Queue (MLFQ); Shortest Job First (SJF), também conhecido em sua forma preemptiva como Shortest Remaining Time, conforme apresentados no escalonamento de processos.

Em sistemas multiprocessados, que possuem múltiplos núcleos de processamento, o escalonamento torna-se ainda mais complexo. O escalonador não precisa apenas decidir qual *thread* será executada, mas também em qual núcleo ela será executada. Um aspecto importante nesse contexto é a afinidade de processador. Muitas implementações procuram manter uma *thread* executando no mesmo núcleo sempre que possível, pois isso permite aproveitar dados que já se encontram armazenados nos níveis mais rápidos da hierarquia de memória cache, como as caches L1 e L2, melhorando o desempenho de execução.

Outro desafio fundamental em sistemas multiprocessados é o balanceamento de carga. O escalonador precisa distribuir as *threads* de forma relativamente uniforme entre os diferentes núcleos disponíveis. Caso muitas *threads* sejam atribuídas a um único núcleo enquanto outros permanecem ociosos, o desempenho global do sistema pode ser significativamente prejudicado. Para evitar esse problema, os sistemas operacionais empregam técnicas de redistribuição periódica de tarefas, migrando *threads* entre núcleos de forma a manter uma utilização equilibrada dos recursos de processamento.

DRAFT